

Project Report

Project Title	Implement Client Script & UI Policy (Incident)
Platform	ServiceNow (Personal Developer Instance)
Team ID	SWTID-2026-2615
Team Members	Sanjana sree M (Team Leader), SRI SADHA S, Mrinalini Baranidharan

1. INTRODUCTION

1.1 Project Overview

Incident Management is one of the most used processes in ServiceNow. The quality of an incident depends on the details entered in the Incident form, but users often skip important fields, enter very short descriptions or forget to add reasons when they put a ticket on hold. This project, **Implement Client Script & UI Policy (Incident)**, uses the client-side features of ServiceNow to guide users while they fill the form. UI Policies control whether fields are mandatory, visible or read-only depending on the State and Priority of the incident, and Client Scripts (onLoad, onChange and onSubmit) display helpful messages and stop the submission of incomplete data. All of this runs in the browser, so users get instant feedback without waiting for the server.

1.2 Purpose

The purpose of the project is to improve the completeness and consistency of incident data and to reduce the effort spent on follow-up questions. The goals are:

- Make the Incident form dynamic, so that fields change according to the State, Priority and Category selected.
- Force users to provide a reason when an incident is put On Hold and resolution details when it is Resolved.
- Give instant on-screen guidance using field messages and information messages.
- Block the submission of Priority 1 incidents that do not have a proper description.
- Gain practical experience with UI Policies, Client Scripts and the g_form client API on a real ServiceNow instance.

2. IDEATION PHASE

2.1 Problem Statement

Incident tickets are frequently saved with missing or poor-quality information. Resolvers then have to contact the caller again, which delays the resolution and affects the service levels. The problem is described from the point of view of the two main users below.

	PS-1	PS-2
I am	a service desk agent who logs and updates incidents	a service desk manager who reviews incident quality
I am trying to	record complete and correct incident details quickly	make sure every incident has enough information to be resolved
But	the form lets me save without a hold reason, resolution notes or a proper description	many tickets arrive with very short or empty descriptions
Because	no rules guide me or stop me when a required field is left empty	fields are optional and there is no check at the time of saving
Which makes me feel	unsure and frustrated when the caller is contacted again	concerned about delays and poor service quality

2.2 Empathy Map Canvas

The empathy map below is for the primary user, the **Service Desk Agent**, who creates and updates incidents on the Incident form.

SAYS • "I do not have time to fill every field." • "Why was this ticket put on hold?" • "The caller needs a quick update."	THINKS • "I hope I have not missed anything." • "A clear form would save me time." • "Incomplete tickets make my work harder."
DOES • Opens the Incident form and fills in the main fields • Changes State, Impact and Urgency • Calls the caller again for missing details	FEELS • Pressured to close tickets quickly • Frustrated by rework • Confident when the form guides them
PAINS • Missing hold reasons and resolution notes • Very short descriptions on critical tickets • No warning before saving wrong data	GAINS • Clear mandatory fields at the right time • Instant messages while typing • Better quality tickets and faster resolution

Fig 2.2 Empathy Map Canvas - Service Desk Agent

2.3 Brainstorming

The team brainstormed ways to improve the Incident form using only client-side features. Each team member listed ideas, and the ideas were then grouped and prioritised by impact and effort.

Idea	Type	Impact	Effort	Decision
Make On hold reason mandatory when State is On Hold	UI Policy	High	Low	Selected
Make Resolution code and notes mandatory when State is Resolved	UI Policy	High	Low	Selected
Make Description mandatory for Priority 1 incidents	UI Policy	High	Low	Selected

Idea	Type	Impact	Effort	Decision
Make key fields read-only for Closed incidents	UI Policy	Medium	Low	Selected
Show a guidance message when a new incident is opened	Client Script (onLoad)	Medium	Low	Selected
Show a field message when Category or Impact changes	Client Script (onChange)	Medium	Low	Selected
Block submit of P1 incident with short description	Client Script (onSubmit)	High	Medium	Selected
Auto-assign groups using GlideAjax	Client + Server	Medium	High	Future scope
Validate data from API and import sets	Data Policy / Business Rule	High	High	Future scope

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The journey map follows the agent from opening the Incident form to saving the record and shows how the solution improves each stage.

Stage	Agent Action	Pain Point (Before)	Experience	Improvement (After)
1. Open form	Clicks Incident > Create New	No guidance on what to fill	Neutral	onLoad script shows a guidance message
2. Enter details	Selects Category, Impact and Urgency	No help on choosing values	Unsure	onChange scripts show field messages
3. Change state	Sets State to On Hold or Resolved	Reason and notes are skipped	Rushed	UI Policies make the fields visible and mandatory
4. Submit	Clicks Submit or Save	Poor data saved for P1 tickets	Worried	onSubmit script blocks invalid data with a message
5. Review	Re-opens a closed incident	Closed records can be edited by mistake	Frustrated	UI Policy makes key fields read-only

3.2 Solution Requirement

Functional Requirements

FR No.	Functional Requirement	Sub Requirement
FR-1	On hold handling	When State is On Hold, show On hold reason and make it mandatory
FR-2	Resolution handling	When State is Resolved, make Resolution code and Resolution notes mandatory
FR-3	Critical incident details	When Priority is 1 - Critical, make Description mandatory
FR-4	Closed incident protection	When State is Closed, make key fields read-only
FR-5	New incident guidance	Show an information message when a new incident form loads
FR-6	Dynamic field messages	Show messages when Category or Impact values change
FR-7	Submit validation	Stop submit if a P1 incident has a description shorter than 20 characters

Non-functional Requirements

NFR No.	Non-functional Requirement	Description
NFR-1	Usability	Messages must be short and clear so that agents understand them at once
NFR-2	Performance	Scripts must run instantly and must not slow the form load noticeably
NFR-3	Reliability	UI Policies must reverse when the condition is no longer true
NFR-4	Maintainability	Scripts must be simple, commented and captured in an Update Set
NFR-5	Security	Access is controlled by roles and ACLs; client checks are only for usability

3.3 Data Flow Diagram

The diagram shows how data moves between the agent, the client-side logic and the Incident table.

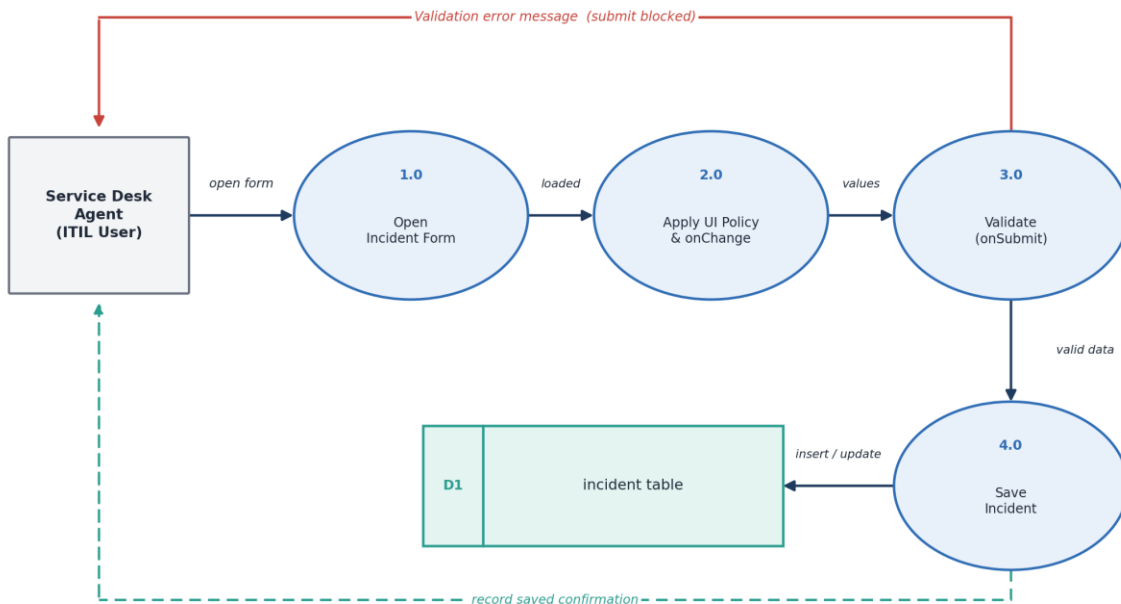


Fig 3.3 Data Flow Diagram (Level 1) - Incident form with UI Policy and Client Scripts

3.4 Technology Stack

Component	Technology / Tool
Platform	ServiceNow (Personal Developer Instance from developer.servicenow.com)
Client-side logic	UI Policies and Client Scripts (onLoad, onChange, onSubmit)
Scripting language	JavaScript using the g_form and g_user client APIs
Data	Incident table (extends Task table)
Server layer	Glide platform, ACLs, Table API (used for reference only)
Deployment / Tracking	Local Update Set
Testing tools	Incident form, JavaScript Log and Field Watcher, browser Developer Tools
Browser	Google Chrome / Microsoft Edge

4. PROJECT DESIGN

4.1 Problem Solution Fit

Problem	Solution	Fit
Hold reason is missing on on-hold tickets	UI Policy: On hold reason visible and mandatory	Every on-hold ticket has a reason
Resolved tickets have no resolution details	UI Policy: Resolution code and notes mandatory	Complete closure information
Critical tickets have poor descriptions	UI Policy and onSubmit script for Priority 1	Enough detail for fast action
Users get no guidance while filling the form	onLoad and onChange Client Scripts	Faster and more accurate entry
Closed tickets can be edited by mistake	UI Policy: read-only fields	Closed records stay reliable

4.2 Proposed Solution

The proposed solution configures four UI Policies and four Client Scripts on the Incident table. UI Policies handle the field behaviour that depends on conditions, and Client Scripts handle messages and validation that need logic. The solution is simple and needs no server-side code or extra plugins, so it is easy to maintain and can be moved to other instances with an Update Set. The flow of the solution is shown below.

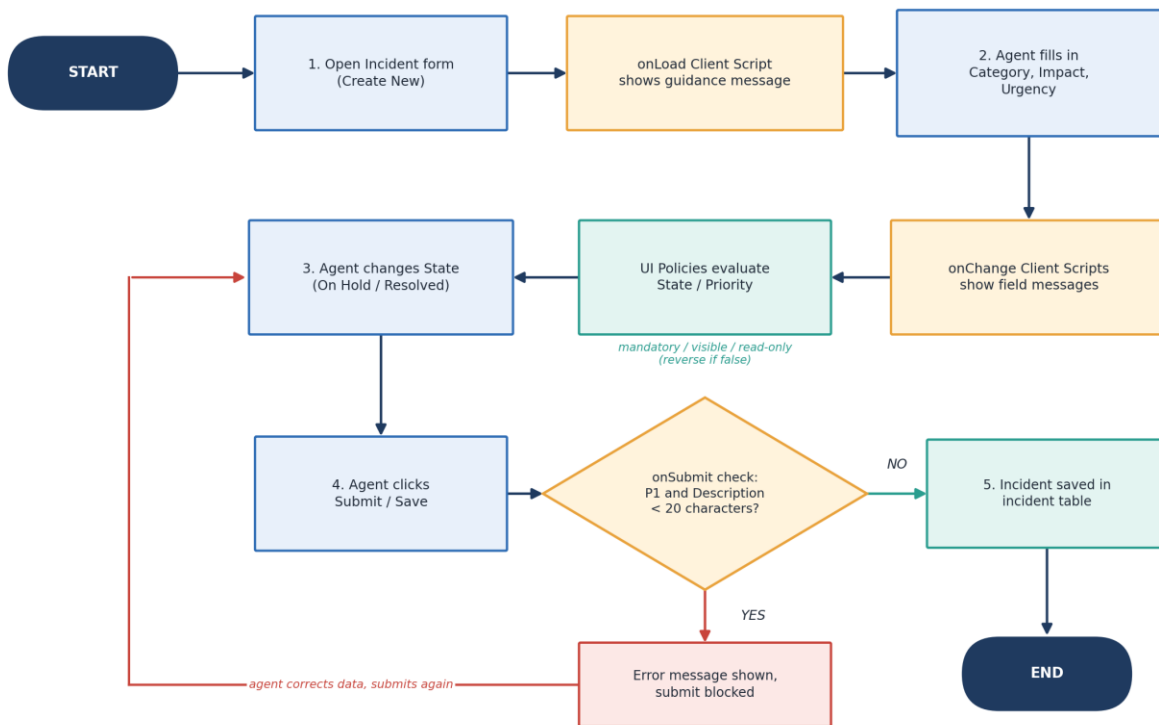


Fig 4.2 Workflow of the proposed solution

UI Policies

ID	Short Description	Condition	UI Policy Actions
UP-1	Incident - On Hold Reason	State is On Hold	On hold reason: Visible = true, Mandatory = true
UP-2	Incident - Resolution Details	State is Resolved	Resolution code: Mandatory; Resolution notes: Mandatory
UP-3	Incident - P1 Description	Priority is 1 - Critical	Description: Mandatory
UP-4	Incident - Closed Read-only	State is Closed	Short description, Category, Description: Read only

All UI Policies have Active, On load and Reverse if false selected, so the form returns to normal when the condition is no longer true.

Client Scripts

ID	Name	Type	Field	Purpose
CS-1	Incident - Welcome Message	onLoad	-	Guidance message for new incidents
CS-2	Incident - Category Guidance	onChange	Category	Field message on Subcategory for Hardware
CS-3	Incident - High Impact Message	onChange	Impact	Reminder to describe business impact
CS-4	Incident - P1 Description Check	onSubmit	-	Block submit if P1 description is under 20 characters

4.3 Solution Architecture

The solution follows the standard ServiceNow layered architecture. The user works in the browser on the Incident form. The UI Policies and Client Scripts form the client-side logic layer, and the validated record is then saved by the server layer in the incident table.

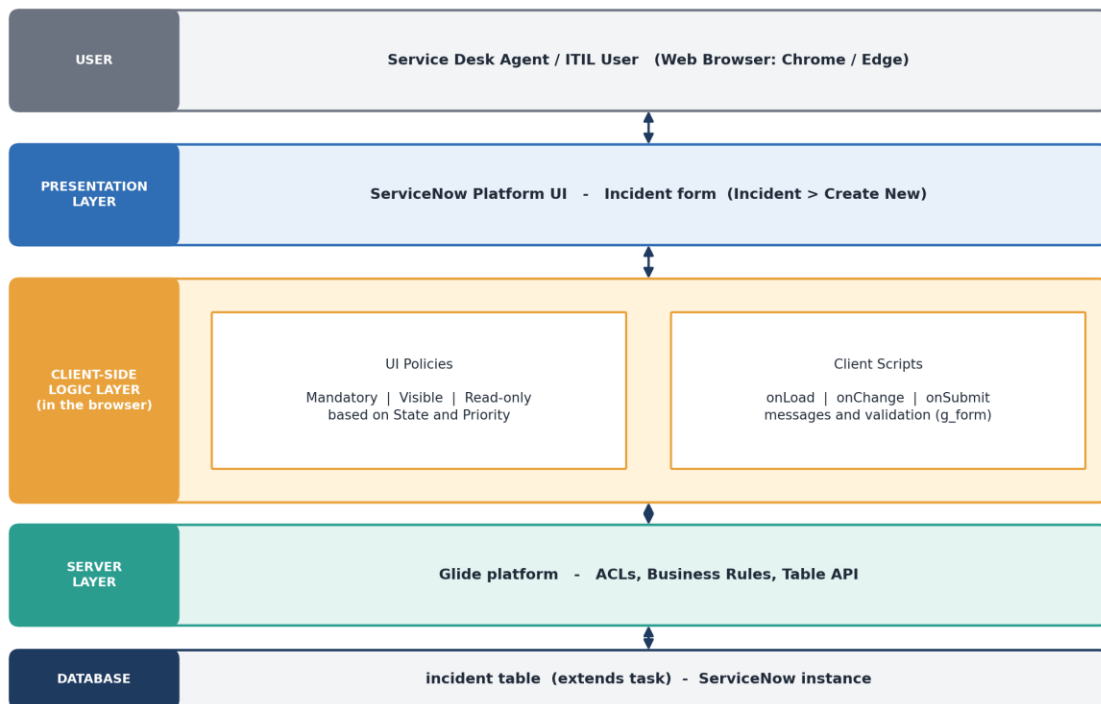


Fig 4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was planned in phases, and the work was divided among the three team members as shown below.

Phase	Activity	Team Member(s)	Deliverable
1	Project kick-off, problem statement, empathy map and brainstorming	All members	Ideation phase
2	Requirement analysis, journey map, PDI setup and Update Set creation	Sanjana sree M, SRI SADHA S	Requirements, working instance
3	Solution design and configuration of the four UI Policies	Sanjana sree M, Mrinalini Baranidharan	UI Policies UP-1 to UP-4
4	Development of the four Client Scripts	SRI SADHA S	Client Scripts CS-1 to CS-4
5	Functional and performance testing, bug fixing and screenshots	Mrinalini Baranidharan, All members	Test results, screenshots
6	Final report, FSD documentation, Update Set export and demo	Mrinalini Baranidharan, Sanjana sree M	Final report and demo

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Each UI Policy and Client Script was tested on the Incident form with a positive case and a negative case. The test cases, expected results and performance checks are given below. The Status column is filled in after the test is executed.

Functional Test Cases

ID	Test Scenario	Steps / Input	Expected Result	Status
TC-1	Welcome message	Open Incident > Create New	Information message is displayed on the form	Pass / Fail
TC-2	On Hold reason	Set State = On Hold	On hold reason appears and is mandatory; save is blocked if empty	Pass / Fail
TC-3	Reverse UI Policy	Change State from On Hold to In Progress	On hold reason is hidden and no longer mandatory	Pass / Fail
TC-4	Resolution details	Set State = Resolved	Resolution code and notes are mandatory	Pass / Fail
TC-5	P1 description mandatory	Set Impact = 1 and Urgency = 1 (Priority 1)	Description becomes mandatory	Pass / Fail
TC-6	Category message	Set Category = Hardware	Field message appears on Subcategory	Pass / Fail
TC-7	Impact message	Set Impact = 1 - High	Field message appears on Impact	Pass / Fail
TC-8	Submit validation (negative)	P1 incident with description of 5 characters, click Submit	Submit is blocked with an error message	Pass / Fail
TC-9	Submit validation (positive)	P1 incident with description of 30 characters, click Submit	Incident is saved successfully	Pass / Fail
TC-10	Closed incident	Open an incident with State = Closed	Short description, Category and Description are read-only	Pass / Fail

Performance Checks

Check	Method	Target	Measured
Incident form load time	Browser Developer Tools (Network tab) or Client Transaction Timings	No noticeable increase with scripts active	____ sec
onChange response time	Change Category / Impact and observe the message	Message appears instantly (under 1 second)	____ sec
onSubmit validation time	Click Submit with invalid data	Error shown instantly without a server call	____ sec
Browser console errors	Developer Tools (Console) and JavaScript Log	No JavaScript errors	Yes / No

7. RESULTS

7.1 Output Screenshots

The screenshots below show the UI Policy configured on the Incident table in the ServiceNow instance.

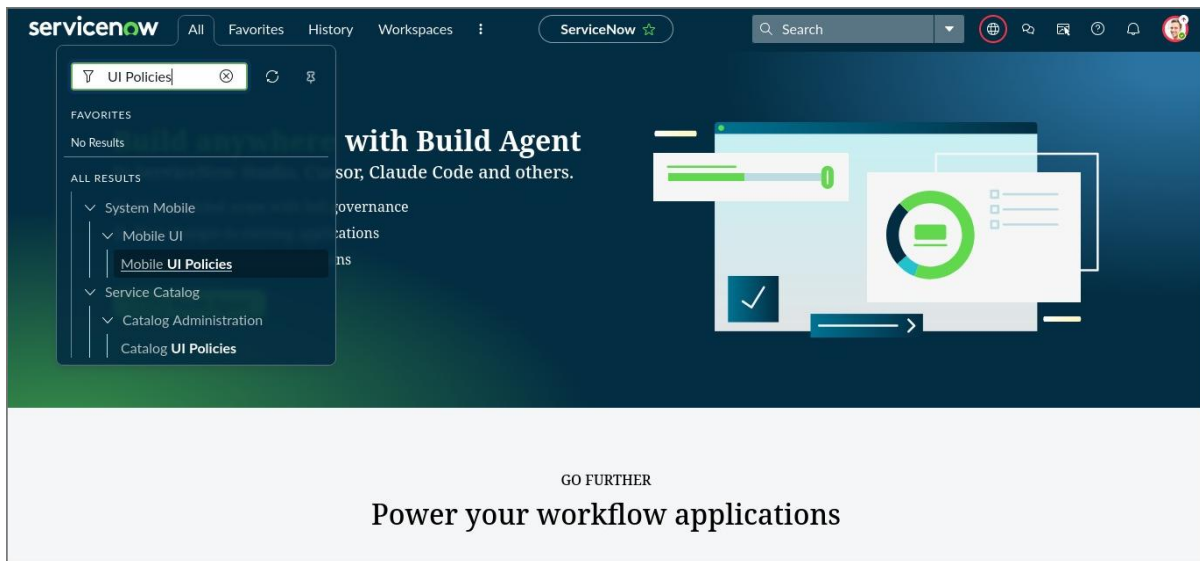


Fig 7.1 Opening UI Policies from the Application Navigator (All > UI Policies)

servicenow

All

Favorites

History

Workspaces

UI Policies

☆

Search

UI Policies

Updated

Search

+

Actions on selected rows...

New

All > Short description >= high impact

Search

Search

Search

Search

Search

Search

Search

High Impact Control

Incident [incident]

impact=1^EQ

true

true

2026-09-29 07:07:55

100

High Impact Control

Incident [incident]

impact=1^EQ

true

true

2026-09-28 23:55:49

100

Weekly Rotations Not All Day

Roster [cmn_rota_roster]

rotation_interval_type=weekly^rotation_a...

false

true

2026-06-13 03:10:31

100

Weekly Rotations All Day

Roster [cmn_rota_roster]

rotation_interval_type=weekly^rotation_a...

false

true

2026-06-13 03:10:31

100

No Reminders

Roster [cmn_rota_roster]

send_reminders=false^EQ

true

true

2026-06-13 03:10:31

100

Show pricing implications checkbox

Question [question]

type=8^ORtype=21^EQ

true

true

2026-06-13 02:56:32

100

Show price if checked

Question [question]

type=7^EQ

true

true

2026-06-13 02:56:32

100

Show Multiple Choices and Choice price

Item Option [sc_cat_item_option]

type=multiple_choice^ORtype=numeric_scal...

true

true

2026-06-13 02:56:32

100

1

to 20 of 1,708

Fig 7.2 UI Policies list filtered for "high impact" on the Incident table

servicenow
All
Favorites
History
Admin
UI Policy - High Impact Control
Search

UI Policy
High Impact Control

This record is in the Global application, but Change Management - Change Schedule is the current application. To edit this record click here.

UI policies change fields on a form based on a set of conditions. Use UI policies to show or hide fields, or to make fields mandatory or read-only based on these conditions. A UI policy specifies one or more actions to take when the policy is triggered. First, create the policy. Then add as many actions as necessary. UI policy actions are applied only if all of the following conditions are met:

- The UI Policy is Active
- The items in the Conditions field evaluate to true
- The field specified in the UI policy action is present on the specified form

[More info](#)

Table Incident [incident]
Application Global
Active

Short description High Impact Control

When to Apply
Conditions impact=1^EQ

Related Links
[Advanced view](#)
[Run Point Scan](#)

Fig 7.3 UI Policy "High Impact Control" on the Incident table with condition impact = 1

servicenow All Favorites History UI Policy Action - assignment_group Search

UI Policy Action assignment_group

This record is in the **Global** application, but **Change Management - Change Schedule** is the current application. To edit this record click [here](#).

There are multiple UI Policies with the same Order for this field - their run order is not predictable: [High Impact Control](#), [Make fields read-only on close](#), [High Impact Control](#)

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

UI policy: **High Impact Control**

Table: Incident [incident]

Field name: **assignment_group**

Application: Global

Mandatory: False

Visible: Leave alone

Read only: False

Clear the field value ☐

Fig 7.4 UI Policy Action for the assignment_group field of "High Impact Control"

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages
- Better quality and completeness of incident data. - Instant feedback in the browser without a server call. - Simple to build with low-code configuration. - Fewer follow-up calls and faster resolution. - Easy to move between instances using an Update Set. - Reversible UI Policies keep the form clean.	- Runs only in the browser UI, not for API, import or background updates. - Can be bypassed by technical users, so server-side rules are still needed. - Too many client scripts can slow down form loading. - Needs separate handling for Workspace and mobile views. - Rules are fixed for the Incident table only. - Does not replace ACLs and Business Rules.

9. CONCLUSION

The project successfully implemented four UI Policies and four Client Scripts on the Incident form of ServiceNow. The form now behaves dynamically: it asks for a hold reason when a ticket is put on hold, asks for resolution details when a ticket is resolved, protects closed records and stops Priority 1 incidents from being saved with a poor description. Users also receive clear messages while they work. The project helped the team understand the difference between UI Policies and Client Scripts, the use of the g_form API, and the importance of tracking changes in Update Sets. The work shows that simple client-side configuration can improve data quality with very little effort.

10. FUTURE SCOPE

- Add server-side Data Policies and Business Rules so that the same checks apply to API, import set and background updates.
- Extend the approach to Problem, Change Request and Service Catalog forms.
- Use Script Includes with GlideAjax for dynamic validation and auto-filling values from other tables.

- Configure the rules for Service Operations Workspace and the mobile application.
- Package the work as a scoped application and add Automated Test Framework (ATF) tests.

11. APPENDIX

Source Code (if any)

Client Script source code used in the project (UI Policies are configured through the form and do not need script).

```
// CS-1  onLoad  |  Table: Incident  |  Name: Incident - Welcome Message
function onLoad() {
    if (g_form.isNewRecord()) {
        g_form.addInfoMessage(
            'Please provide complete details: caller, category, impact and urgency.');
```

```
// CS-2  onChange  |  Field: Category  |  Name: Incident - Category Guidance
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') { return; }
    g_form.hideFieldMsg('subcategory');
    if (newValue == 'hardware') {
        g_form.showFieldMsg('subcategory', 'Select the type of hardware issue.', 'info');
```

```
// CS-3  onChange  |  Field: Impact  |  Name: Incident - High Impact Message
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') { return; }
    if (newValue == '1') {
        g_form.showFieldMsg('impact',
            'High impact: describe the business impact in the Description.', 'info');
```

```
// CS-4  onSubmit  |  Table: Incident  |  Name: Incident - P1 Description Check
function onSubmit() {
    var priority = g_form.getValue('priority');
    var description = g_form.getValue('description');
    if (priority == '1' && description.trim().length < 20) {
        g_form.addErrorMessage(
            'For a Priority 1 incident, Description must be at least 20 characters.');
```

Dataset Link

Not applicable. The project uses the Incident table records created in the ServiceNow Personal Developer Instance and does not use an external dataset.